Python Exercise — Dijkstra's Shortest Path Algorithm

Jiwon Kim, The University of Queensland

2026

Contents

Python Exercise — Dijkstra's Shortest Path Algorithm

1. Background

Dijkstra's algorithm finds the **shortest path from an origin node r to every other node** in a network. It uses two vectors to store results:

- **Label vector $\mathbf{L}^r$**: L_i^r is the shortest path cost from origin r to node i. The origin is initialised to $L_r^r = 0$; all others start at ∞.
- **Backnode vector $\mathbf{q}^r$**: q_i^r is the node immediately before i on the shortest path from r. All entries are initialised to -1; the origin always remains -1.

The algorithm also maintains Q, the set of *unvisited* nodes. At each iteration it picks the unvisited node with the smallest current label, removes it from Q, and updates the labels of its downstream neighbours. When Q is empty, the shortest paths from r to all reachable nodes are finalised.

Once $\mathbf{q}^r$ is known, the shortest path to any destination can be reconstructed by tracing backnodes from the destination back to the origin.

Terminology note

In this exercise, the two endpoints of a directed link (i, j) are referred to as:

- `init_node` — the **initial** (upstream/tail) node i
- `term_node` — the **terminal** (downstream/head) node j

instead of `from_node` / `to_node` as in Week 3.

Similarly, the dictionary that maps each node to its list of directly reachable downstream nodes is called the **forward star** — written `forward_star` in code. This replaces the `downstream_nodes` name used in Week 3. We adopt this naming convention from now on.

2. The Network

This exercise uses the following small road network with **4 nodes** and **5 directed links**:

Link	`init_node`	`term_node`	`length` (km)
(1, 2)	1	2	1.0
(1, 3)	1	3	1.2
(3, 2)	3	2	1.5
(3, 4)	3	4	1.1
(4, 2)	4	2	1.0

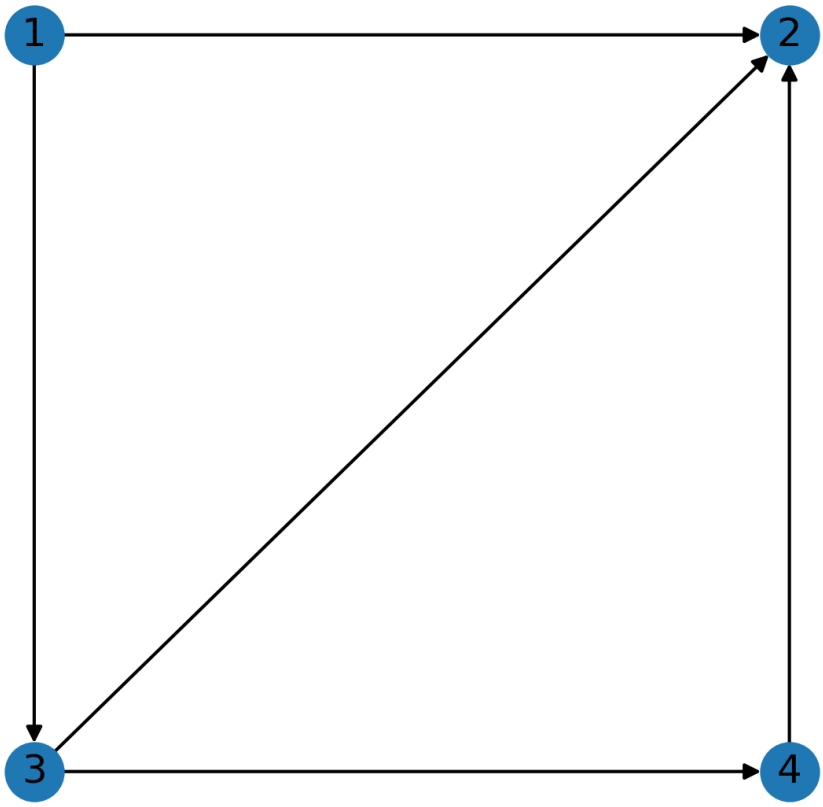

Figure 1: Road network

---

3. Starting Point

The code below builds the two DataFrames for this exercise: `df_nodes` and `df_links`. `df_nodes` has one row per node, with columns for the node ID and (x, y) coordinates. `df_links` has one row per directed link, with columns for the upstream node (`init_node`), downstream node (`term_node`), and link length.

```
import pandas as pd
import numpy as np

# Tells NumPy to use the legacy print format, which prints plain numbers instead of verbose ty
np.set_printoptions(legacy='1.25')

# node_data has three columns: the node ID, and x/y coordinates for plotting.
node_data = {
    'node': [1, 2, 3, 4],
    'x':    [0, 1, 0, 1],
    'y':    [1, 1, 0, 0],
}
# Convert the node_data dictionary into a DataFrame.
df_nodes = pd.DataFrame(node_data)
# Set the node ID as the index for easy lookup by node ID later.
df_nodes.set_index('node', inplace=True, drop=False)
```

```
# link_data has three columns: the upstream node (init_node), the downstream
# node (term_node), and the link length in km.
link_data = {
    'init_node': [1, 1, 3, 3, 4],
    'term_node': [2, 3, 2, 4, 2],
    'length':    [1.0, 1.2, 1.5, 1.1, 1.0],
}
# Convert the link_data dictionary into a DataFrame.
df_links = pd.DataFrame(link_data)
# Set a MultiIndex on df_links for easy lookup by (init_node, term_node).
df_links.set_index(['init_node', 'term_node'], inplace=True, drop=False)

# Print the link and node DataFrames to check they look correct.
print("--- Network Data ---")
print("df_nodes:")
print(df_nodes)

print("\ndf_links:") # \n adds a blank line before the next print for readability
print(df_links)
```

Output:

```
df_nodes:
      node  x  y
node
1        1  0  1
2        2  1  1
3        3  0  0
4        4  1  0

df_links:
                     init_node  term_node  length
init_node term_node
1         2                  1          2     1.0
          3                  1          3     1.2
3         2                  3          2     1.5
          4                  3          4     1.1
4         2                  4          2     1.0
```

The forward star is also built here as a starting point, using the same pattern as `downstream_nodes` from Week 3:

```
# Construct the forward star dictionary to store the downstream nodes for each node.
forward_star = {}

# Loop through all nodes in df_nodes to initialise forward_star with empty lists.
for row in df_nodes.itertuples(index=False):
    forward_star[row.node] = []

# Loop through all links in df_links to populate the forward_star dictionary.
for row in df_links.itertuples(index=False):
    forward_star[row.init_node].append(row.term_node)

print("\nforward_star:", forward_star)
```

Output:

```
forward_star: {1: [2, 3], 2: [], 3: [2, 4], 4: [2]}
```

---

4. Task 1 — Add a Cost Column

In this exercise, the **cost** of traversing each link is defined as cost = length × 2.

Add a new column `'cost'` to `df_links` and fill it with the computed cost. Print `df_links` when done.

You can do this in two ways:

- *Create the cost column first with some initial values (e.g. `0.0`), then fill it row-by-row using `itertuples()`.*
- *Or Directly assign the result of a column operation on `df_links['length']` multiplied by 2.*

Expected output:

```
--- Link Data with Costs ---
                  init_node  term_node  length  cost
init_node term_node
1         2               1          2     1.0   2.0
          3               1          3     1.2   2.4
3         2               3          2     1.5   3.0
          4               3          4     1.1   2.2
4         2               4          2     1.0   2.0
```

---

5. Task 2 — Initialise the Algorithm

Before the main loop, set up the three data structures required by Dijkstra's algorithm. Use `df_nodes` to iterate over all nodes.

- `L` (dictionary): shortest path cost from origin to each node. Set every entry to `np.inf`, then set `L[orig_node] = 0.0`.
- `q` (dictionary): backnode of each node on the current shortest path. Set every entry to `-1`.
- `Q` (list): the set of unvisited nodes. Add every node to this list.

Use `orig_node = 1` as the origin. Print `orig_node`, `L`, `q`, and `Q` after initialisation.

Expected output:

```
--- Algorithm Initialisation ---
orig_node: 1
L: {1: 0.0, 2: inf, 3: inf, 4: inf}
q: {1: -1, 2: -1, 3: -1, 4: -1}
Q: [1, 2, 3, 4]
```

---

6. Task 3 — Dijkstra's Main Loop

Implement the main loop of Dijkstra's algorithm. Repeat the following until `Q` is empty:

1. Find the node $i \in Q$ with the smallest $L[i]$.
2. Remove i from `Q`.
3. For each downstream node j of i that is **still in `Q`**:
   - Look up the link cost t_{ij} using `df_links.at[(i, j), 'cost']`.
   - Compute the candidate cost: newL $= L[i] + t_{ij}$.
   - If newL $< L[j]$: update $L[j] \leftarrow$ newL and $q[j] \leftarrow i$.

Print the final `L` and `q` after the loop.

Use the following skeleton as a starting point:

```
# Repeat while there are still unvisited nodes.
while len(Q) > 0:

    # Step 1: find the node in Q with the smallest label.
    i = -1
    minL = np.inf
    # TODO: loop over Q to find the node with the minimum label

    # Step 2: mark i as visited by removing it from Q.
    Q.remove(i)

    # Step 3: for each downstream node j of i still in Q,
    # look up the link cost, compute the candidate label,
    # and update L[j] and q[j] if improved.
    # TODO: loop over forward_star[i] and update L[j] and q[j]

print("\n--- Algorithm Results ---")
print("L:", L)
print("q:", q)
```

Expected output:

```
--- Algorithm Results ---
L: {1: 0.0, 2: 2.0, 3: 2.4, 4: 4.6}
q: {1: -1, 2: 1, 3: 1, 4: 3}
```

---

7. Task 4 — Reconstruct the Shortest Paths

Use the backnode vector `q` to reconstruct the shortest path from the origin to each reachable destination. Store all paths in a dictionary called `odpath` where:

- Each **key** is a tuple `(orig_node, dest_node)`
- Each **value** is a list of node IDs forming the shortest path (origin → destination)

For each destination node:

1. Use `continue` to skip if it is the origin or unreachable (`L[dest_node] == np.inf`).
2. Initialise `path = [dest_node]`, which starts with the destination node and will be built backwards by prepending nodes.

3. Trace back through `q`, prepending each backnode to `path`, until the origin is reached (`q[current] == -1`).
4. Store the completed path in `odpath` with key `(orig_node, dest_node)`.

Print each path and its cost.

Use the following skeleton as a starting point:

```
# Initialise an empty dictionary to store the shortest paths.
odpath = {}

for row in df_nodes.itertuples(index=False):
    dest_node = row.node
    # TODO: use 'continue' to skip if dest_node is the origin or unreachable

    # Initialise the path with the destination node, and set the current node to dest_node.
    path = [dest_node]
    current = dest_node

    # TODO: while-loop - trace back through q, inserting each backnode of the current node to

    # Store the completed path in odpath with key (orig_node, dest_node).
    odpath[(orig_node, dest_node)] = path

print("\n--- Shortest Paths ---")
for od, path in odpath.items():
    print(f"{od}: {path}  (cost: {L[od[1]]})")
```

Expected output:

```
--- Shortest Paths ---
(1, 2): [1, 2]  (cost: 2.0)
(1, 3): [1, 3]  (cost: 2.4)
(1, 4): [1, 3, 4]  (cost: 4.6)
```

---

8. Reflection Questions

1. Change `orig_node` to `3` and re-run all cells from Task 2 onwards. What are the resulting `L`, `q`, and shortest paths?
2. Now change `orig_node` to `2` and re-run. You will see that nothing is printed under `--- Shortest Paths ---`. Why? How would you modify the printing code to display a helpful message when no paths are found?